Alcance del Proyecto

Motor de Procesamiento Distribuido para Análisis de Riesgo en Contratación Pública

1. Problema que resuelve el sistema

El análisis de patrones de riesgo en la contratación pública colombiana, particularmente la detección de procesos con baja competencia real combinada con sobreprecios relativos y concentración de proveedores, es una tarea computacionalmente intensiva. Los conjuntos de datos de SECOP II contienen más de 9 millones de procesos de contratación y millones de contratos electrónicos, acumulando datos de varios gigabytes. Analizar la correlación entre estas tres dimensiones mediante un único hilo de ejecución se vuelve impráctico: los tiempos de cómputo crecen linealmente con el volumen de datos y el proceso queda limitado por la memoria RAM disponible en una sola máquina.

El sistema resuelve este problema mediante un motor de procesamiento distribuido, construido desde cero y sin apoyarse en frameworks de terceros, que implementa el paradigma MapReduce sobre una arquitectura Master-Worker orquestada por Kubernetes. Los conjuntos de datos de entrada se particionan en bloques durante la Etapa 1, donde los Jobs A, B1 y B2 procesan de forma independiente y en paralelo el índice de competencia, la desviación de precio y la concentración proveedor-entidad respectivamente. Durante la Etapa 2, el Master coordina un reduce-side join sobre los resultados agregados de la Etapa 1, produciendo un indicador conjunto de riesgo que identifica procesos, proveedores y entidades de alto riesgo, junto con métricas de rendimiento.

En síntesis, el sistema busca reducir el tiempo de análisis de riesgo mediante paralelización real en múltiples nodos físicos, ofrecer tolerancia a fallos de nodos Worker durante la ejecución de un trabajo tanto a nivel de tarea como a nivel de proceso, servir como banco de pruebas controlado para medir el comportamiento de un join distribuido, y producir un indicador de valor público reproducible sobre cualquier período del histórico de SECOP II.

2. No-objetivos

El sistema explícitamente no busca reemplazar a organismos de control como la Contraloría o la Procuraduría. El indicador de riesgo generado es una señal preliminar para investigación y vigilancia posterior, no una conclusión definitiva sobre irregularidades. El análisis implementado utiliza criterios estadísticos y de concentración, no metodologías de auditoría formal.

El sistema no implementa orquestación dinámica de recursos más allá de lo que provee Kubernetes de forma nativa. La topología es fija, con un Master y tres Workers definidos en los manifiestos de despliegue; no hay autoescalado del número de nodos físicos ni incorporación de nodos en caliente al clúster. La lógica de particionamiento, scheduling de tareas y reasignación de trabajo ante fallos es responsabilidad exclusiva del motor propio y no se delega en primitivas nativas de Kubernetes como un Job con completions indexados, ya que esa lógica es precisamente el objeto de aprendizaje del proyecto.

El sistema no provee alta disponibilidad a nivel de estado del Master. Kubernetes reinicia automáticamente el contenedor del Master si este falla, mediante su Deployment y sus probes de salud, pero el estado del trabajo en curso reside en memoria y no se replica; no se implementa failover con preservación de estado del nodo maestro. Tampoco incluye autenticación, autorización ni cifrado en tránsito: la comunicación entre nodos opera sobre una red confiable y cerrada, sin control de acceso.

El sistema no es una plataforma de procesamiento en tiempo real ni de streaming. Los trabajos se ejecutan en modo batch: ambos conjuntos de datos se cargan, se procesan y se entrega un resultado consolidado al finalizar. Tampoco incluye interfaz gráfica de usuario; la interacción se realiza vía API o línea de comandos. Finalmente, el sistema no garantiza tolerancia ante partición de red: se asume una red físicamente estable durante la ejecución de un trabajo, y los escenarios de partición parcial quedan fuera del alcance de las pruebas de tolerancia a fallos.

3. Restricciones de red

Las cuatro máquinas del clúster están conectadas físicamente mediante Ethernet, sobre una red local cerrada dedicada al clúster. Al tratarse de una LAN cableada entre los cuatro hosts en el mismo segmento, se estima una latencia punto a punto menor o igual a 1 ms para el tráfico gRPC de control. Este valor es un estimado inicial de diseño y debe validarse empíricamente una vez desplegado el clúster.

Se asume un enlace Gigabit Ethernet por nodo, sin garantía de ancho de banda dedicado si la red llegara a compartirse con otro tráfico. El protocolo de comunicación es gRPC sobre HTTP/2, lo que implica sensibilidad tanto a la latencia de establecimiento de conexión como a la pérdida de paquetes en el plano de control, donde viajan los heartbeats y la reasignación de chunks. La red interna del clúster Kubernetes, resuelta mediante el Service headless de los workers, se asume estable durante la ejecución de un trabajo; el mecanismo de tolerancia a fallos del motor propio cubre caídas de proceso o de nodo Worker, no fallos de red prolongados o intermitentes.

4. Hardware disponible por nodo

El clúster está compuesto por cuatro máquinas físicas con hardware idéntico: un nodo Master y tres nodos Worker.

Especificación	**Valor por nodo**
Rol	1 Master, 3 Workers, hardware idéntico entre todos los nodos
CPU	20 núcleos
RAM	64 GB
Almacenamiento	100 GB
Red	Ethernet, interfaz cableada dedicada al clúster
Contenedorización	Docker, cada nodo ejecuta su rol en contenedor

Especificación	Valor por nodo
Orquestación	Kubernetes: Deployment para el Master, StatefulSet con Service headless para los Workers, podAntiAffinity para garantizar un worker por nodo físico

Con los cuatro nodos idénticos, la capacidad agregada del clúster es de 80 núcleos, 256 GB de RAM y 300 GB de almacenamiento en total. El Master, además de coordinar la asignación de chunks y consolidar los resultados en la fase Reduce, comparte el mismo perfil de hardware que los Workers, por lo que no constituye un cuello de botella de cómputo: su rol es principalmente de coordinación y agregación, no de procesamiento intensivo de datos.

5. Volumen de datos esperado y tamaño de dataset soportado

Los datos de entrada corresponden a dos conjuntos de datos públicos de SECOP II: Procesos de Contratación con 9,14 millones de filas y Contratos Electrónicos con aproximadamente 6 millones de filas. El tamaño total estimado en disco es de 20 a 40 GB en formato crudo sin comprimir, muy por debajo de la capacidad de almacenamiento del clúster.

El procesamiento se ejecuta en dos etapas. En la Etapa 1, tres Jobs se ejecutan de forma independiente y en paralelo: el Job A calcula el índice de competencia real sobre Procesos de Contratación, el Job B1 calcula la desviación de precio agrupando por categoría UNSPSC sobre Contratos Electrónicos, y el Job B2 calcula la concentración proveedor-entidad agrupando por la relación proveedor-entidad, también sobre Contratos Electrónicos. En la Etapa 2, el Master coordina un reduce-side join sobre las tres tablas intermedias resultantes de la Etapa 1, no sobre el dato crudo, manejando explícitamente como left outer join el caso de procesos sin contrato asociado, los cuales se conservan en el resultado final como hallazgo propio en lugar de descartarse.

El tamaño de cada chunk individual en la Etapa 1 es configurable por el Master; se recomienda mantenerlo en el orden de cientos de MB, muy por debajo del límite de RAM por nodo, para permitir balanceo de carga fino, reasignación rápida ante fallos y transmisión por streaming de gRPC en lugar de mensajes únicos de gran tamaño. Este volumen de datos se encuentra dentro de la capacidad real del clúster y deberá revisarse si el volumen crece en fases posteriores.

6. Métricas de éxito

6.1. Throughput mínimo esperado

El sistema debe sostener un speedup mínimo de 2x frente al procesamiento secuencial equivalente al ejecutar sobre los tres Workers, como umbral mínimo para justificar la arquitectura distribuida frente a un enfoque secuencial. Se define como meta un throughput agregado mínimo de 50 MB/s de datos procesados por el clúster completo, medido como el tamaño total del dataset dividido entre el execution_time_ms reportado en el FinalResult. Dado que la Etapa 1 y la Etapa 2 tienen perfiles de carga distintos, agregación simple frente a join distribuido, este umbral y sus mediciones reales se reportan por separado para cada etapa. Estos números son objetivos iniciales a validar empíricamente; el proyecto contempla medir y reportar el speedup y el throughput reales obtenidos como parte de sus objetivos específicos, no solo cumplir un umbral fijo.

6.2. Tiempo máximo de recuperación ante fallo

Ante la caída de un nodo Worker durante la ejecución de un trabajo, se distinguen dos mecanismos de recuperación que operan en planos distintos. A nivel de tarea, el Master debe detectar el fallo, mediante la ausencia del heartbeat o el timeout de la tarea, en un máximo de 10 segundos, y reasignar el chunk afectado a otro Worker disponible en un máximo adicional de 5 segundos; en conjunto, el tiempo máximo de recuperación de la tarea debe ser menor o igual a 15 segundos. A nivel de proceso, Kubernetes reinicia el pod del worker caído según sus livenessProbe y readinessProbe, en un tiempo típicamente mayor a la recuperación de la tarea, dado que el intervalo de verificación de los probes es independiente del heartbeat del motor propio; ambos tiempos se miden y reportan por separado. La caída del nodo Master se considera fuera de alcance de recuperación automática con preservación de estado, según lo indicado en la sección de no-objetivos: aunque Kubernetes reinicia el contenedor del Master, el trabajo en curso se pierde.

6.3. Corrección del indicador de riesgo

Los indicadores calculados en la Etapa 1 y el join de la Etapa 2 deben ser verificables mediante comparación contra una ejecución de referencia secuencial: para un subconjunto de procesos elegido aleatoriamente, el índice de competencia real, la desviación de precio y la concentración proveedor-entidad calculados por el sistema distribuido deben coincidir exactamente con los calculados por un script Python de referencia ejecutado sobre los mismos datos. Esta validación incluye explícitamente el caso de procesos sin contrato asociado, verificando que se conserven en el resultado final en lugar de descartarse. Esto valida la corrección del algoritmo independientemente del rendimiento.